

■ ASR Voice Assistant

Project File Structure & Component Reference

Generated: June 21, 2026 • Version 1.0.0 • Python 3.12

Modules	10
Source Files	50+
Test Suites	5
Config Files	8
Docker Files	7

Table of Contents

1. Root Level Files

- › audio.py
- › main.py
- › models.py
- › first.py
- › whiper_test.py
- › requirement.txt
- › .env
- › install.cmd
- › Makefile
- › pyproject.toml
- › tox.ini
- › sample-0.mp3

2. src/voice_assistant/ — Main Package

- › __init__.py

3. src/voice_assistant/asr/ — Automatic Speech Recognition

- › transcriber.py
- › processor.py
- › exceptions.py
- ›

4. src/voice_assistant/audio/ — Audio I/O & Processing

- › audio_processor.py
- › buffer_manager.py
- › input_handler.py
- › output_handler.py
- › vad.py
- › exceptions.py

5. src/voice_assistant/core/ — Core Orchestration & Config

- › orchestrator.py
- › config.py
- › constants.py

6. src/voice_assistant/nlu/ — Natural Language Understanding

- › intent_classifier.py
- › entity_extractor.py
- › preprocessor.py
- › exceptions.py

7. src/voice_assistant/tts/ — Text-to-Speech

- › synthesizer.py
- › processor.py
- › exceptions.py

8. src/voice_assistant/response_generation/ — Response Engine

- › llm_engine.py
- › template_engine.py
- › formatter.py

- › exceptions.py
- 9. **src/voice_assistant/storage/** — Data Persistence
 - › database.py
 - › encryption.py
 - › exceptions.py
 - ›
- 10. **src/voice_assistant/tasks/** — Task Execution System
 - › executor.py
 - › registry.py
 - ›
 - › exceptions.py
- 11. **src/voice_assistant/wake_word/** — Wake Word Detection
 - › detector.py
 - › utils.py
- 12. **src/voice_assistant/api/** — REST API Layer
 - › server.py
 - › schemas.py
 - › middleware.py
 - › exceptions.py
 - ›
- 13. **src/voice_assistant/utls/** — Utilities
 - › logger.py
 - › helpers.py
 - › validators.py
 - › decorators.py
 - › metrics.py
 - › device_apis.py
 - › exceptions.py
- 14. **config/** — YAML Configuration Files
 - › base.yaml
 - › development.yaml
 - › production.yaml
 - › testing.yaml
 - ›
 - ›
- 15. **docker/** — Containerisation
 - › Dockerfile
 - › Dockerfile.dev
 - › docker-compose.yml
 - › entrypoint.sh
- 16. **tests/** — Test Suite
 - › conftest.py
 - ›
 - ›
 - ›
 - ›
- 17. **monitoring/** — Observability Stack
 - ›
 - ›

>
>

18. docs/ — Project Documentation

- > FOLDER_STRUCTURE.md
- > COMPLETE_GUIDE.md
- > API_DOCUMENTATION.md
- > INSTALLATION.md
- > PHASE1_SETUP.md
- > project_analysis.md

Complete Project Tree

```
ASR/
■■■ audio.py ← Real-time STT demo (RealtimeSTT)
■■■ main.py ← FastAPI server entry point
■■■ models.py ← Whisper model manager
■■■ first.py ← Scratch playground script
■■■ whiper_test.py ← Whisper quick-test
■■■ requirement.txt ← Python dependencies
■■■ .env ← Environment variables (secrets)
■■■ install.cmd ← Windows setup script
■■■ Makefile ← Build / dev automation
■■■ setup.py/cfg/pyproject.toml ← Package build config
■■■ sample-0.mp3 ← Test audio sample
■
■■■ src/voice_assistant/
■ ■■■ __init__.py ← Package init + lazy imports
■ ■■■ asr/
■ ■ ■■■ transcriber.py ← Whisper STT engine
■ ■ ■■■ processor.py ← Audio pre-processor
■ ■ ■■■ exceptions.py ← ASR exceptions
■ ■ ■■■ models/ ← ASR data models
■ ■■■ audio/
■ ■ ■■■ audio_processor.py ← General audio processing
■ ■ ■■■ buffer_manager.py ← Circular audio buffer
■ ■ ■■■ input_handler.py ← Microphone capture
■ ■ ■■■ output_handler.py ← Speaker playback
■ ■ ■■■ vad.py ← Voice activity detection
■ ■ ■■■ exceptions.py
■ ■■■ core/
■ ■ ■■■ orchestrator.py ← VoiceAssistant (Phase 2)
■ ■ ■■■ config.py ← Config manager (Phase 2)
■ ■ ■■■ constants.py ← System constants
■ ■■■ nlu/
■ ■ ■■■ intent_classifier.py
■ ■ ■■■ entity_extractor.py
■ ■ ■■■ preprocessor.py
■ ■ ■■■ exceptions.py
■ ■■■ tts/
■ ■ ■■■ synthesizer.py ← Text-to-Speech engine
■ ■ ■■■ processor.py ← TTS post-processor
■ ■ ■■■ exceptions.py
■ ■■■ response_generation/
■ ■ ■■■ llm_engine.py ← LLM response gen
■ ■ ■■■ template_engine.py ← Template responses
■ ■ ■■■ formatter.py ← Speech-friendly formatter
■ ■ ■■■ exceptions.py
■ ■■■ storage/
■ ■ ■■■ database.py ← SQLAlchemy ORM models
■ ■ ■■■ encryption.py ← AES-256 encryption
■ ■ ■■■ exceptions.py
■ ■ ■■■ migrations/ ← Alembic migrations
■ ■■■ tasks/
■ ■ ■■■ executor.py ← Task dispatcher
■ ■ ■■■ registry.py ← Handler registry
■ ■ ■■■ handlers/ ← Concrete task handlers
■ ■ ■■■ exceptions.py
■ ■■■ wake_word/
■ ■ ■■■ detector.py ← openWakeWord detector
■ ■ ■■■ utils.py
```

```
■ ■■■ api/
■ ■■■ server.py ← FastAPI app factory
■ ■■■ schemas.py ← Pydantic schemas
■ ■■■ middleware.py ← Auth / rate-limit / logging
■ ■■■ exceptions.py
■ ■■■ routers/ ← Route modules
■ ■■■ utils/
■ ■■■ logger.py / helpers.py / validators.py
■ ■■■ decorators.py / metrics.py / device_apis.py
■ ■■■ exceptions.py
■
■ ■■■ config/
■ ■■■ base.yaml / development.yaml / production.yaml
■ ■■■ staging.yaml / testing.yaml
■ ■■■ logging/ ■■■ models/
■
■ ■■■ docker/
■ ■■■ Dockerfile / Dockerfile.dev / Dockerfile.test
■ ■■■ docker-compose.yml / docker-compose.prod.yml
■ ■■■ entrypoint.sh
■
■ ■■■ tests/
■ ■■■ conftest.py
■ ■■■ unit/ integration/ e2e/ performance/ fixtures/
■
■ ■■■ monitoring/
■ ■■■ prometheus/ grafana/ logging/ tracing/
■
■ ■■■ docs/
■ ■■■ FOLDER_STRUCTURE.md / COMPLETE_GUIDE.md
■ ■■■ API_DOCUMENTATION.md / INSTALLATION.md
■ ■■■ project_analysis.md (+ PDFs)
```

Root Level Files

audio.py	DEMO
<i>Real-Time Speech Recognition Entry Point</i> Demonstrates live microphone-to-text transcription using the RealtimeSTT library. Initialises an AudioToTextRecorder with the Whisper 'medium' model on CUDA (int8). Enters an infinite loop: calls recorder.text() to block until a sentence is completed, then prints the transcription. Intended as a quick-start demo for real-time ASR.	
main.py	SERVER
<i>FastAPI Application Entry Point (Phase 1 API Server)</i> Core REST API server built with FastAPI. Defines Pydantic models (TranscriptionRequest, TranscriptionResponse, HealthCheckResponse, ModelInfoResponse). Provides endpoints: GET /health, GET /api/v1/status, POST /api/v1/transcribe (file upload + transcription), GET /api/v1/model-info, GET /api/v1/supported-formats, GET /api/v1/languages. Handles CORS middleware, lifespan events (model load on startup, unload on shutdown), and global exception handlers. Launched via Uvicorn when run directly.	
models.py	CORE
<i>Whisper Model Manager</i> Defines WhisperModelManager: manages loading, caching, and introspection of OpenAI Whisper models. Supports 5 sizes: tiny (39M), base (140M), small (244M), medium (769M), large (1.55B). Reads config from environment variables (MODEL_SIZE, DEVICE, LANGUAGE). Provides load_model(), unload_model(), get_model_info(), get_device_info(), list_downloaded_models(). Module-level singleton via initialize_model_manager() / get_model_manager().	
first.py	SCRATCH
<i>Scratch / Playground Script</i> A minimal scratch script (6 lines). Iterates over a small list and conditionally prints elements. Not part of the production system — used for quick algorithm exploration.	
whiper_test.py	TEST
<i>Whisper Quick-Test Script</i> A brief standalone test script for verifying Whisper installation and basic transcription. Useful for sanity-checking the environment before running the full API server.	
requirement.txt	CONFIG
<i>Python Dependency Manifest</i> Lists all Python packages required: openai-whisper, torch/torchaudio, librosa, scipy (audio/ML); fastapi, uvicorn, pydantic, python-multipart (web API); pyaudio, numpy (audio I/O); python-dotenv, pyyaml (config); redis, sqlalchemy, alembic (storage); cryptography, loguru, requests (utilities); pytest, black, flake8 (dev tools).	
.env	CONFIG
<i>Environment Configuration (secrets & settings)</i>	

Stores environment variables: API host/port, model size and device, language, upload size limits, allowed audio formats, feature flags (caching, sentiment analysis, NER), Redis URL, database URI, log level, CORS origins.

install.cmd

SETUP

Windows Installation Script

Batch script automating environment setup on Windows: creates a virtual environment, installs PyTorch with CUDA, installs all requirements, downloads Whisper models offline, and verifies the installation.

Makefile

DEVOPS

Build & Dev Automation

Defines make targets for common tasks: install, run-dev, run-prod, test, lint, format, docker-build, docker-up, clean. Simplifies developer workflow on Unix/WSL environments.

setup.py / setup.cfg / pyproject.toml

BUILD

Python Package Build Configuration

Configures the voice_assistant package for distribution. Defines package metadata (name, version, author), entry points, classifiers, install_requires, and optional extras (dev, gpu, full).

MANIFEST.in / mkdocs.yml / tox.ini

CONFIG

Documentation & Testing Config

MANIFEST.in: controls which files are included in source distributions. mkdocs.yml: MkDocs documentation site configuration. tox.ini: multi-environment test runner configuration (pytest, flake8, mypy).

sample-0.mp3

ASSET

Test Audio Sample

A sample MP3 audio file (~122 KB) used to manually test transcription endpoints and verify the Whisper model produces correct output.

src/voice_assistant/ — Main Package

src/voice_assistant/__init__.py

CORE

Package Initialiser & Lazy Import Registry

Top-level package file. Uses a `__getattr__` hook for lazy loading of all major classes (VoiceAssistant, Config, Transcriber, AudioProcessor, WakeWordDetector, TextToSpeech, etc.) to prevent circular imports. Exports `__version__ = '1.0.0'`, `__all__`, `get_version()`, and `get_package_info()`.

src/voice_assistant/asr/ — Automatic Speech Recognition

asr/transcriber.py

ASR

Whisper-Based Speech-to-Text Transcriber

Core transcription engine. Defines `WhisperModel` enum (TINY→LARGE_TURBO) and `TranscriptionResult` dataclass (text, confidence, language, duration, segments). Transcriber class: auto-device detection, `_load_model()`, `transcribe()` (file or numpy array), `batch_transcribe()`, `transcribe_with_timestamps()`, `_calculate_confidence()` using `no_speech_prob`. Context manager support (with statement). Module-level `transcribe_audio()` convenience function.

asr/processor.py

ASR

ASR Audio Pre-Processor

Handles pre-processing of raw audio before transcription: resampling to 16 kHz, normalisation, noise reduction, and chunking of long audio into manageable segments.

asr/exceptions.py

ASR

ASR Custom Exceptions

Defines the ASR exception hierarchy: `ASRException` (base), `ModelLoadError`, `TranscriptionError`, `UnsupportedFormatError`, `AudioTooShortError`, `LanguageDetectionError`. Used throughout the ASR module for structured error handling.

asr/models/

ASR

ASR Data Models Directory

Contains dataclass/Pydantic models specific to the ASR pipeline: segment models, language detection results, and model configuration schemas.

src/voice_assistant/audio/ — Audio I/O & Processing

audio/audio_processor.py

AUDIO

Audio Processing Pipeline

General-purpose audio processing: loading audio files, resampling, mono conversion, normalisation, trimming silence, and converting between formats (WAV, MP3, FLAC). Uses `librosa` and `scipy` under the hood.

audio/buffer_manager.py

AUDIO

Audio Buffer Manager

Thread-safe circular buffer for streaming audio. Manages audio chunks from the microphone, implements read/write with blocking, and provides utilities for flushing and resizing the buffer.

audio/input_handler.py

AUDIO

Microphone Input Handler

Manages `PyAudio` microphone streams. Opens audio streams with configurable sample rate, channels, and chunk size. Provides `start_recording()`, `stop_recording()`, and `read_chunk()` for real-time audio capture.

audio/output_handler.py

AUDIO

Speaker Output Handler

Handles audio playback via PyAudio. Plays TTS-generated audio to the system speakers, manages playback state, and provides volume control utilities.

audio/vad.py

AUDIO

Voice Activity Detector (VAD)

Implements WebRTC-based voice activity detection using webrtcvad. Detects speech vs. silence in 10/20/30 ms frames. Used to trigger ASR only when the user is speaking, preventing wasteful transcription of silence.

audio/exceptions.py

AUDIO

Audio Custom Exceptions

AudioException hierarchy: DeviceNotFoundError, StreamOpenError, BufferOverflowError, PlaybackError. Provides clear error messages for audio hardware issues.

src/voice_assistant/core/ — Core Orchestration & Config

core/orchestrator.py

CORE

Main Voice Assistant Orchestrator (placeholder)

Will contain the VoiceAssistant class — the top-level coordinator that ties together wake word detection → ASR → NLU → task execution → TTS response into a single pipeline. Currently a placeholder (empty file); implementation is planned for Phase 2.

core/config.py

CORE

Configuration Manager (placeholder)

Will expose a Config class for loading YAML/env configuration into typed settings objects. Planned to support from_file(), from_env(), and validation. Currently empty.

core/constants.py

CORE

System-Wide Constants

Centralised constants: default sample rates, supported audio formats, model size thresholds, timeout values, API version string, and environment variable names.

src/voice_assistant/nlu/ — Natural Language Understanding

nlu/intent_classifier.py

NLU

Intent Classification Engine

Classifies user utterances into intents (e.g., play_music, set_alarm, answer_question). Will integrate rule-based matching and/or a small transformer model for classification.

nlu/entity_extractor.py

NLU

Named Entity Extractor

Extracts structured entities (time, date, location, person names, numbers) from transcribed text. Provides slots that task handlers use to parameterise actions.

nlu/preprocessor.py

NLU

NLU Text Pre-Processor

Cleans and normalises transcribed text before classification: lowercasing, punctuation removal, contraction expansion, and tokenisation.

nlu/exceptions.py

NLU

NLU Custom Exceptions

NLUException, IntentNotFoundError, EntityExtractionError for structured NLU error handling.

src/voice_assistant/tts/ — Text-to-Speech

tts/synthesizer.py

TTS

TTS Synthesizer

Converts text responses to audio waveforms. Supports multiple TTS backends (pyttsx3 offline, Coqui TTS, or cloud APIs). Returns audio as numpy arrays or WAV bytes for playback via the audio output handler.

tts/processor.py

TTS

TTS Audio Post-Processor

Post-processes synthesised audio: adjusts speed, pitch, and volume; applies effects; and converts to the output format expected by AudioOutputHandler.

tts/exceptions.py

TTS

TTS Custom Exceptions

TTSEnction (base), SynthesisError, VoiceNotFoundError, BackendUnavailableError.

src/voice_assistant/response_generation/ — Response Engine

response_generation/llm_engine.py

AI

LLM Response Generator

Interfaces with a local or remote LLM (e.g., LLaMA, Gemini, OpenAI) to generate natural language responses for complex queries that go beyond simple task handlers.

response_generation/template_engine.py

AI

Template-Based Response Generator

Generates fast, deterministic responses from Jinja2 templates for common intents (e.g., 'The time is {time}', 'Setting alarm for {time}'). Used when LLM latency is unacceptable.

response_generation/formatter.py

AI

Response Formatter

Formats raw LLM or template output into polished, speech-friendly text: removes markdown, expands abbreviations, and adds natural pauses.

response_generation/exceptions.py

AI

Response Generation Exceptions

ResponseGenerationError, TemplateNotFoundError, LLMLLTimeoutError.

src/voice_assistant/storage/ — Data Persistence

storage/database.py

DB

SQLAlchemy Database Models

Defines the ORM base and ConversationLog table (columns: id, timestamp, user_text, intent, entities JSON, response, processing_time_ms). Used for conversation history, audit logging, and analytics.

storage/encryption.py

DB

Data Encryption Utilities

Provides AES-256 encryption/decryption for sensitive stored data (conversation logs, user preferences). Uses the cryptography library with Fernet symmetric encryption.

storage/exceptions.py

DB

Storage Custom Exceptions

StorageException, DatabaseConnectionError, EncryptionError, MigrationError.

storage/migrations/

DB

Alembic Database Migrations

Contains Alembic migration scripts for evolving the database schema. Run via alembic upgrade head to apply all pending migrations.

src/voice_assistant/tasks/ — Task Execution System

tasks/executor.py

TASK

Task Executor

Receives an intent + entities from the NLU pipeline and dispatches to the appropriate task handler. Manages task timeouts and error recovery.

tasks/registry.py

TASK

Task Handler Registry

A registry mapping intent names to handler classes. Supports dynamic registration of custom handlers via `@register_task` decorator.

tasks/handlers/

TASK

Concrete Task Handlers Directory

Contains individual handler modules for each supported intent: e.g., `AlarmHandler`, `MusicHandler`, `WeatherHandler`, `TimerHandler`, `DeviceControlHandler`.

tasks/exceptions.py

TASK

Task Custom Exceptions

`TaskException`, `HandlerNotFoundError`, `TaskTimeoutError`, `TaskExecutionError`.

src/voice_assistant/wake_word/ — Wake Word Detection

wake_word/detector.py

WAKE

Wake Word Detector (openWakeWord)

Live implementation: opens a PyAudio 16 kHz mono stream, reads 1280-frame chunks, runs `openWakeWord` prediction on each chunk, and prints detections when score > 0.1. Loaded model: 'hey_mycroft'. Handles clean shutdown on Ctrl+C.

wake_word/utils.py

WAKE

Wake Word Utilities

Helper functions: model path resolution, audio chunk validation, sensitivity tuning utilities, and performance benchmarking helpers.

src/voice_assistant/api/ — REST API Layer

api/server.py

API

FastAPI Application Factory

Creates and configures the FastAPI app instance: registers routers, applies middleware (CORS, auth, rate-limiting), and sets up exception handlers.

api/schemas.py

API

Pydantic Request/Response Schemas

Defines all API-level Pydantic models: TranscriptionRequest, TranscriptionResponse, StatusResponse, ErrorResponse. Ensures strict input validation and OpenAPI docs generation.

api/middleware.py

API

Custom API Middleware

Implements request logging middleware (logs method, path, status code, latency), API key authentication middleware, and rate-limiting per client IP.

api/exceptions.py

API

API-Layer Exception Handlers

Registers FastAPI exception handlers for HTTPException, ValidationError, and unhandled errors. Returns standardised JSON error envelopes with timestamp and error code.

api/routers/

API

API Router Modules Directory

Contains FastAPI APIRouter files split by resource: transcription.py, health.py, models.py. Each router handles a subset of API endpoints and is registered in server.py.

src/voice_assistant/utils/ — Utilities

utils/logger.py

UTIL

Logging Configuration

Configures loguru or stdlib logging with rotating file handlers, coloured console output, and structured JSON log format for production.

utils/helpers.py

UTIL

General Helper Functions

Miscellaneous utilities: file format detection, safe path construction, duration formatting, retry decorator, and async-safe file I/O helpers.

utils/validators.py

UTIL

Input Validators

Validates audio files (format, size, duration), language codes, and API parameters. Raises structured ValidationError on invalid input.

utils/decorators.py

UTIL

Reusable Decorators

Common decorators: @retry(n), @timer (logs execution time), @require_model (checks model loaded), @async_timeout.

utils/metrics.py	UTIL
<i>Performance Metrics Collector</i> Collects and exposes Prometheus metrics: transcription latency histogram, error rate counter, model load time gauge, and active request gauge.	
utils/device_apis.py	UTIL
<i>Device API Integrations</i> Wrappers for smart-home device APIs: light control, thermostat, media player. Used by task handlers to execute physical device commands.	
utils/exceptions.py	UTIL
<i>Shared Utility Exceptions</i> Base UtilityException and sub-classes: ValidationError, TimeoutError, DeviceAPIError.	

config/ — YAML Configuration Files

config/base.yaml	CONFIG
<i>Base Configuration (shared defaults)</i> Defines default values for all settings: model size, device, sample rate, log level, API prefix, and feature flags. Extended by environment-specific files.	
config/development.yaml	CONFIG
<i>Development Environment Overrides</i> Enables debug logging, hot-reload, mock TTS, and disables auth for local development.	
config/production.yaml	CONFIG
<i>Production Environment Overrides</i> Enables auth, Redis caching, rate limiting, and structured JSON logging.	
config/staging.yaml / testing.yaml	CONFIG
<i>Staging & Testing Configs</i> Staging: production-like with relaxed auth. Testing: uses in-memory SQLite, mock audio I/O, and smallest Whisper model.	
config/logging/	CONFIG
<i>Logging Configuration Directory</i> YAML files configuring logging handlers, formatters, and levels per environment.	
config/models/	CONFIG
<i>Model Configuration Directory</i>	

Per-model YAML files specifying compute type, batch size, and language hints.

docker/ — Containerisation

docker/Dockerfile

DOCKER

Production Docker Image

Multi-stage build: Python 3.11-slim base, installs CUDA runtime, copies source, installs requirements, and sets CMD to launch Uvicorn.

docker/Dockerfile.dev

DOCKER

Development Docker Image

Adds development tools, mounts source as volume for hot-reload.

docker/docker-compose.yml

DOCKER

Docker Compose (dev stack)

Orchestrates services: voice_assistant API, Redis (caching), PostgreSQL (storage), Prometheus (metrics), Grafana (dashboards).

docker/entrypoint.sh

DOCKER

Container Entry Point Script

Runs DB migrations (alembic upgrade head), then starts the Uvicorn server.

tests/ — Test Suite

tests/conftest.py

TEST

Pytest Fixtures & Shared Configuration

Defines shared pytest fixtures: mock audio files, FastAPI test client, mock Whisper model, temp directories, and database session fixture.

tests/unit/

TEST

Unit Tests

Isolated unit tests for each module: transcriber, audio_processor, vad, intent_classifier, entity_extractor, tts_synthesizer, etc.

tests/integration/

TEST

Integration Tests

Tests end-to-end pipelines: file upload → transcription → response. Uses real Whisper (tiny model).

[tests/e2e/](#)

TEST

End-to-End Tests

Full pipeline tests including API calls, database writes, and TTS playback.

[tests/performance/](#)

TEST

Performance & Benchmark Tests

Measures transcription latency, throughput, and model load time under load.

monitoring/ — Observability Stack

[monitoring/prometheus/](#)

OPS

Prometheus Configuration

prometheus.yml: scrape configs for the ASR API, Redis, and Postgres exporters.

[monitoring/grafana/](#)

OPS

Grafana Dashboards

Pre-built JSON dashboard: request rate, transcription latency p50/p99, error rate, GPU utilisation.

[monitoring/logging/](#)

OPS

Centralised Log Config

Filebeat/Loki config for shipping container logs to a central log aggregator.

[monitoring/tracing/](#)

OPS

Distributed Tracing

OpenTelemetry/Jaeger config for end-to-end request tracing across services.

docs/ — Project Documentation

[docs/FOLDER_STRUCTURE.md](#)

DOCS

Folder Structure Reference

Detailed annotated tree of the entire project hierarchy with purpose notes.

[docs/COMPLETE_GUIDE.md](#)

DOCS

Developer Complete Guide

End-to-end guide: setup, configuration, API usage, extending the system, and deployment.

[docs/API_DOCUMENTATION.md](#)

DOCS

REST API Reference

Full OpenAPI-style docs for all endpoints: request/response schemas, examples, error codes.

docs/INSTALLATION.md

DOCS

Installation Guide

Step-by-step setup for Windows/Linux/macOS, GPU configuration, and offline model download.

docs/PHASE1_SETUP.md

DOCS

Phase 1 Implementation Notes

Technical details of Phase 1 (offline ASR): design decisions, model benchmarks, known issues.

docs/project_analysis.md

DOCS

Project Analysis Document

Requirements analysis, architecture overview, technology choices, and roadmap for all phases.

Technology Stack Summary

Layer	Technology	Purpose
Speech-to-Text	OpenAI Whisper	Offline, multi-language ASR (tiny→large-v3)
Real-Time STT	RealtimeSTT + faster-whisper	Live microphone transcription
Wake Word	openWakeWord	Hands-free activation ('hey mycroft')
Web API	FastAPI + Uvicorn	REST endpoints, WebSocket (Phase 2)
Data Validation	Pydantic v2	Request/response schema validation
Audio I/O	PyAudio + librosa	Microphone capture & audio processing
VAD	webrtcvad-wheels	Voice activity detection
Database ORM	SQLAlchemy + Alembic	Conversation logs & migrations
Caching	Redis	Model output caching (Phase 2)
Encryption	cryptography (Fernet)	AES-256 data-at-rest encryption
Metrics	Prometheus + Grafana	Performance monitoring
Containerisation	Docker + Compose	Deployment & dev environments
Testing	Pytest + pytest-asyncio	Unit, integration & E2E tests
Code Quality	Black + Flake8	Formatting & linting

Voice Assistant Pipeline (Phase 2 Target)

```
[ Microphone ] → VAD → Wake Word Detector
↓
[ Audio Buffer ] → Audio Processor → ASR Transcriber (Whisper)
↓
[ NLU: Preprocessor → Intent Classifier → Entity Extractor ]
↓
[ Task Executor → Handler Registry → Concrete Task Handler ]
↓
[ Response Generator (LLM / Template) → Formatter ]
↓
[ TTS Synthesizer → Audio Output Handler → Speaker ]
↓
[ Storage: ConversationLog → Database ]
```

This document was auto-generated from the live project source code.